

ML Pipeline Design & MLOps Analysis

Credit Default Risk Scoring, continued from Assignment 1

Course DDM501 — AI in DevOps, DataOps, MLOps
Assignment Individual Assignment 2 (5%)
Student Võ Minh Sang — 25MS13286

Recap of Assignment 1. A consumer finance company needs a real-time service that estimates each credit card applicant's probability of defaulting next month. The service maps that probability to APPROVE (< 0.30), REVIEW ($0.30\text{--}0.60$) or DECLINE (≥ 0.60). The lab implementation has an Airflow training pipeline tracked in MLflow; the proposed serving architecture would load the @champion alias through a FastAPI service monitored with Prometheus and Grafana. The primary metric is ROC AUC (target ≥ 0.75), with PR AUC, recall at the 0.30 threshold, Brier score and a **fairness gate** (selection-rate gap between sexes ≤ 0.10).

Refinements in this document: (1) Lab 2 CLI training, evaluation and the Airflow DAG use the shared prepare_features function before the sklearn Pipeline; this assignment's experiment script calls the same underlying add_derived_features function. Lab 4 moves feature derivation *inside* its Pipeline. (2) Validation is an explicit three-level gate before training. (3) The registry uses aliases instead of deprecated stages. (4) The Lab 2 CLI registers rejected candidates by default for an audit trail, while the DAG skips registration after a failed quality gate. The experiment table below comes from code/run_experiments.py and its saved results.

1. Pipeline Design

1.1 Pipeline architecture

Data flow: raw frame → validation → 29 engineered columns → fitted Pipeline → metrics → quality gate → registry decision

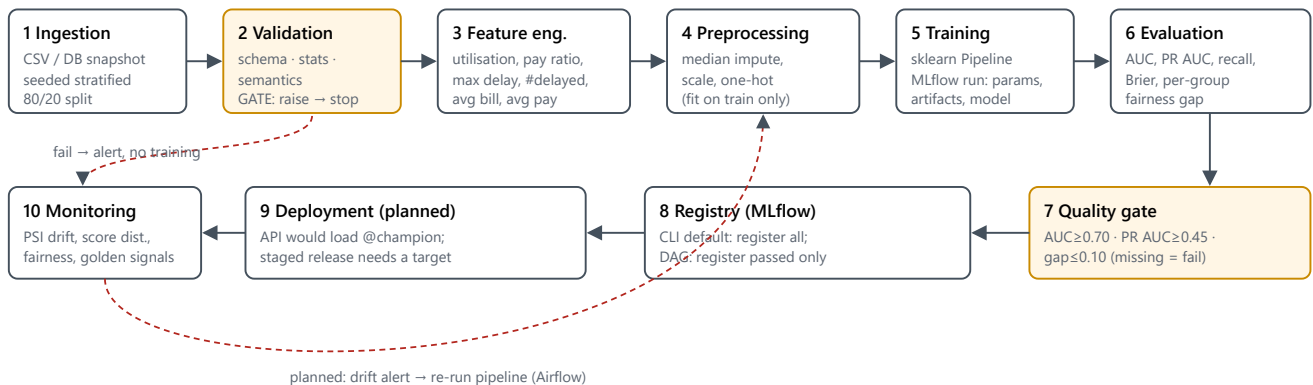


Figure 1. Pipeline architecture. Amber boxes are gates that can stop the run. Preprocessing (impute, scale, one-hot) is fitted inside the scikit-learn Pipeline on the training fold only. Feature engineering is the shared prepare_features function applied before the Pipeline in both training and evaluation; the Lab 4 version moves it into the Pipeline as a FunctionTransformer.

1.2 Stage specifications

Stage	Purpose	Input → Output	Key operations	Quality gate
Ingestion	A reproducible snapshot and split	CSV/DB → frame; X/y train/test; stats	load_raw, stratified train_test_split(random_state=501), dataset_stats	30,000 rows × 24 columns; train/test default rates within 1 pp
Validation	Stop broken data before it becomes a model	Raw frame → report dict (raises on error)	Schema (23 features + target, numeric), statistics ($\geq 5,000$ rows, $\leq 2\%$ missing, 5–60% positive rate), semantics (codes, AGE 18–100, PAY $-2..8$, PAY_AMT ≥ 0)	Any error raises DataValidationError; the DAG task fails and training never runs
Feature engineering	Encode analyst knowledge	23 raw → 29 columns	utilisation = mean(BILL)/LIMIT; payment_ratio = PAY_AMT1/BILL_AMT1 (clipped to 0–5); max_delay; n_months_delayed; avg_bill; avg_pay	Returns a copy; divides by replace(0, NaN), so there is never an inf
Preprocessing	Numerically sound model inputs	Feature frame → matrix	ColumnTransformer: one-hot (handle_unknown="ignore") for SEX/EDUCATION/MARRIAGE; median impute + StandardScaler for the rest	Returned <i>unfitted</i> , then fitted on the training fold only (no leakage)
Training	Fit and record the run	Features, labels, params → Pipeline, run_id	Model families LogReg / RF / HGB; log params, data stats, validation report, feature list, model with signature	The run is tagged data_validation=passed
Evaluation	Measure quality and fairness	Model, test split → metrics dict	ROC AUC and PR AUC (on probabilities); precision, recall and F1 at 0.30 (on labels); Brier; confusion counts; per-group metrics; gap	Groups with $n < 50$ or only one class are skipped
Registry / deployment	Decide what serving would load	run_id, metrics → version or skip	By default, the CLI calls promote_model and registers every candidate; --no- register skips this step. The DAG registers only after its gate passes. A passing candidate beats the champion at an AUC margin of 0.002 or becomes a challenger.	The CLI tags a failed candidate <i>rejected</i> with no alias. The DAG branches to skip_promotion, registers nothing and leaves the reasons in the quality_gate XCom.

1.3 Design rationale

- **Validation before training, not just after ingestion.** The expensive and risky step is training and promotion. A degenerate target (all zeros) still trains "successfully" and scores 100% accuracy. Placing the gate directly upstream of training guarantees that no model is produced from data that failed a check, and its report travels with the run as an artifact.
- **Modularity and reuse.** Core validation, feature derivation and evaluation functions live in the pipeline/ package. The CLI runner and Airflow DAG reuse them; this assignment's sweep uses the same underlying feature derivation. Tracking and orchestration still have side effects and separate wrappers.
- **Error handling and recovery.** Validators return lists of errors, so one run reports every problem at once. Airflow retries transient failures once after 2 minutes. A failed task stops everything downstream. The cleanup task uses none_failed_min_one_success, so it runs on either branch. The champion alias is changed only after the quality gate passes; an MLflow logging failure can still leave a partial run to inspect.
- **Scalability.** Stages exchange files on a shared volume and pass only metadata through XCom, so they can move to separate workers (LocalExecutor now, CeleryExecutor or KubernetesExecutor later). The prototype has 30,000 rows, and the tested HistGradientBoosting runs trained on 24,000 rows in under a second here. A larger dataset would require new memory and runtime measurements before choosing Spark or another architecture.

2. Experiment Tracking & Metrics Analysis

2.1 Experiment design

Variables: algorithm family (Dummy, LogisticRegression, RandomForest, HistGradientBoosting); hyperparameters (C, depth, trees, learning rate, L2); features (23 raw vs 29 with the engineered features); preprocessing and imbalance handling (`class_weight="balanced"`). **Baseline:** E0, which predicts the training prior for everyone. Its ROC AUC of 0.50 is the no-skill reference, and E1 (logistic regression on raw features) is the simple model that any complex model must beat.

ID	Family	Features	Key hyperparameters	Question it answers
E0	Dummy (prior)	raw	strategy=prior	What does "no skill" score?
E1	LogReg	raw	C=1.0	Simple linear baseline
E2	LogReg	derived	C=1.0	Do engineered features help a linear model?
E3	LogReg	derived	C=0.1	Does stronger regularisation matter?
E4	LogReg	derived	C=1.0, class_weight=balanced	Effect of reweighting the minority class
E5	RandomForest	derived	200 trees, depth 8, leaf 20	Shallow bagging ensemble
E6	RandomForest	derived	300 trees, depth 12, leaf 20	Deeper forest
E7	HistGB	raw	300 iter, lr 0.06, depth 6, L2 1.0	Do trees need engineered features?
E8	HistGB	derived	300 iter, lr 0.06, depth 6, L2 1.0	Default production candidate
E9	HistGB	derived	500 iter, lr 0.03, depth 8, L2 2.0	Slower learning, more regularisation

2.2 Metrics strategy

- **Primary: ROC AUC.** It is independent of the threshold and measures ranking quality. The business applies its own thresholds, so ranking is what the model owes it.
- **Secondary:** PR AUC (sensitive to the 23% minority class, which is where the losses are); precision and recall at 0.30 (the REVIEW threshold: recall is the share of future defaulters stopped, precision the share of flagged applicants who really default); Brier score (the thresholds act on raw probabilities, so calibration matters); training time and per-row latency (operating cost).
- **Guardrail: fairness gap.** The maximum difference in selection rate ($\text{score} \geq 0.30$) between SEX groups. It is a hard gate at 0.10.
- **Link to the business.** Recall at 0.30 maps to the *loss reduction* goal, the flag rate $(\text{TP} + \text{FP})/N$ maps to the *manual review workload* goal, and the fairness gap maps to the *compliance* goal. A 0.01 gain in AUC matters only if it does not push the review rate or the fairness gap past their limits.

2.3 Code: MLflow set-up, logging and registration

```
# pipeline/training.py – experiment set-up and one tracked run
def setup_mlflow(tracking_uri: str = MLFLOW_TRACKING_URI,
                 experiment_name: str = MLFLOW_EXPERIMENT_NAME) -> str:
    """Point MLflow at the tracking store and make sure the experiment exists."""
    mlflow.set_tracking_uri(tracking_uri)          # file store locally, http://mlflow:5000 in Docker
    mlflow.set_experiment(experiment_name)
    return tracking_uri

with mlflow.start_run(run_name=run_name) as run:
    mlflow.log_param("model_type", model_type)
    mlflow.log_param("n_features", len(feature_columns))
    mlflow.log_param("random_state", RANDOM_STATE)
    for key, value in params.items():              # every hyper-parameter
        mlflow.log_param(key, value)
    mlflow.log_dict(validation_report, "validation_report.json")    # why the data was trusted
    mlflow.set_tag("data_validation", "passed" if validation_report["passed"] else "failed")
    mlflow.log_dict({"features": feature_columns, "feature_columns.json"}) # what serving must send
    pipeline = build_pipeline(model_type, feature_columns, **params).fit(X_train, y_train)
    mlflow.sklearn.log_model(sk_model=pipeline, artifact_path="model",
                             input_example=X_train.head(3).astype("float64")) # signature

# pipeline/evaluation.py – metrics logged back into the same run
with mlflow.start_run(run_id=run_id):
    mlflow.log_metrics({"roc_auc": ..., "pr_auc": ..., "recall": ..., "brier": ...})
    mlflow.log_metric("fairness_gap", gap)
    mlflow.log_dict(result, "evaluation.json")      # includes per-group metrics
```

```
# pipeline/registry.py – register, gate, alias (aliases replace deprecated stages)
def promote_model(run_id: str, metrics: Dict[str, float],
                  model_name: str = "credit-default-classifier") -> Dict[str, Any]:
    client = MlflowClient()
    gate = passes_quality_gate(metrics)            # missing metric => fail (fail closed)
    version = mlflow.register_model(f"runs://{run_id}/model", model_name).version
    if not gate["passed"]:
        outcome = "rejected"                       # registered for the audit trail, no alias
    elif beats_champion(metrics, model_name, margin=0.002):
        outcome = "champion"
        client.set_registered_model_alias(model_name, "champion", version)
    else:
        outcome = "challenger"
        client.set_registered_model_alias(model_name, "challenger", version)
    client.set_model_version_tag(model_name, version, "quality_gate",
                                "passed" if gate["passed"] else "failed")
    client.set_model_version_tag(model_name, version, "outcome", outcome)
    return {"version": version, "outcome": outcome, "quality_gate": gate}

# serving loads by alias, never by a hard-coded version number
model = mlflow.sklearn.load_model("models:/credit-default-classifier@champion")
```

Versioning convention. There is one registered model per use case (credit-default-classifier), with versions numbered automatically. The alias @champion is what serving loads and @challenger is the next candidate. The tags quality_gate, outcome and failed_checks record why each version was or was not promoted. Every version links back to its run, and so to its parameters, data statistics, validation report and feature list.

2.4 Results

Held-out test set: 6,000 rows, 23.3% positive. The decision threshold is 0.30. Latency is the batch-scoring time divided by the number of rows.

ID	ROC AUC	PR AUC	Precision	Recall	F1	Brier	Fairness gap	Train (s)	µs / row	Gate
E0 prior	0.5000	0.2328	0.000	0.000	0.000	0.1786	0.0000	0.03	0.7	fail
E1 logreg raw	0.7449	0.5448	0.537	0.503	0.520	0.1457	0.0543	0.05	0.7	pass
E2 logreg derived	0.7511	0.5534	0.533	0.507	0.520	0.1440	0.0544	0.06	0.8	pass
E3 logreg C=0.1	0.7511	0.5534	0.531	0.507	0.519	0.1440	0.0558	0.08	0.8	pass
E4 logreg balanced	0.7505	0.5528	0.288	0.896	0.436	0.1917	0.0810	0.05	0.8	pass
E5 RF depth 8	0.7482	0.5393	0.564	0.483	0.520	0.1452	0.0324	0.54	6.9	pass
E6 RF depth 12	0.7502	0.5406	0.554	0.486	0.518	0.1450	0.0352	0.93	10.4	pass
E7 HGB raw	0.7481	0.5461	0.566	0.480	0.520	0.1447	0.0333	0.22	1.4	pass
E8 HGB derived	0.7473	0.5444	0.556	0.482	0.516	0.1448	0.0306	0.18	1.4	pass
E9 HGB slow	0.7475	0.5435	0.559	0.485	0.520	0.1449	0.0276	0.36	2.4	pass

2.5 Analysis

- **Every trained model beats the baseline by a wide margin** (AUC about +0.25, PR AUC more than doubles), but the families differ by at most 0.006 AUC on this test split. No uncertainty interval or paired significance test was calculated, so this run alone does not establish that one family ranks risks better. Other metrics matter.
- **Derived features help logistic regression on this split.** The ratios lift its ROC AUC from 0.745 to 0.751 (E1 → E2). The tested HGB configuration did not improve with them (E7 → E8); tree splits may already capture part of their effect. Changing C from 1.0 to 0.1 made no meaningful difference in these test metrics.
- **class_weight="balanced" changes the operating point, not the ranking.** E4 has the same AUC as E2, but its probabilities are no longer calibrated (Brier 0.19 against 0.14). At the 0.30 threshold it flags most applicants: recall 0.90, precision 0.29. That would roughly triple the manual review queue and widen the fairness gap to 0.081. Recall should be adjusted by moving the threshold, not by breaking calibration.
- **Accuracy vs fairness.** The most accurate family (logistic regression, gap 0.054–0.056) treats the two groups least evenly. The tree ensembles cut the gap roughly in half (0.028–0.035) for at most 0.004 AUC. All models pass the 0.10 gate, so the gate alone does not decide; the business priority does.
- **Cost.** Every candidate scores in under 11 µs per row, far inside the 100 ms budget. Random forests are the slowest to train and score, and their model files are the largest.

Recommendation for a future registry decision: E9 (HistGradientBoosting on engineered features, lr 0.03, depth 8, L2 2.0) as @champion, with E2 (logistic regression) as @challenger. E9 has the narrowest fairness gap (0.028, about half of E2's). Its AUC is 0.0036 below E2 on this split; no uncertainty test was run. Its Brier score (0.145) is close to E2's (0.144), while its precision and recall reflect a different trade-off. Explainability would need to be implemented and verified separately. E2 is a useful challenger candidate because it is simpler; the experiment script tracks runs but does not assign registry aliases.

3. Workflow Orchestration Design

3.1 DAG architecture

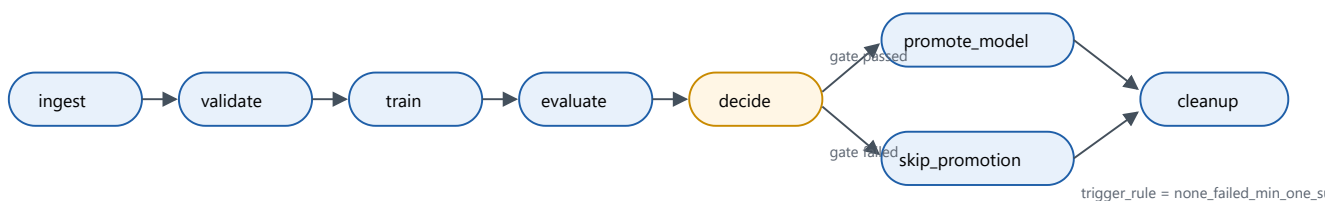


Figure 2. The credit_default_training DAG. decide is a BranchPythonOperator; the side that is not chosen is marked skipped.

Task	Description	XCom out (metadata only)
ingest	Load the raw data, split it, write <code>split.joblib</code> and <code>raw.parquet</code> to the per-run directory on the shared volume.	<code>run_dir</code> , <code>data_stats</code>
validate	Three-level validation with <code>raise_on_error=True</code> ; write <code>validation_report.json</code> .	<code>validation_report</code>
train	Fit the Pipeline inside an MLflow run named <code>airflow-{ds}</code> ; dump <code>model.joblib</code> .	<code>mlflow_run_id</code>
evaluate	Metrics and per-group metrics, logged to the same run; full JSON to disk.	<code>metrics</code> (scalars only)
decide	Apply <code>passes_quality_gate</code> ; return the <code>task_id</code> of the next task.	<code>quality_gate</code>
promote_model	Register the model, compare with the champion, set the alias.	<code>promotion</code>
skip_promotion	EmptyOperator that marks the rejected path.	—
cleanup	Delete the run directory on either branch.	—

Dependencies. The chain is strictly linear up to `decide`, because each task consumes the previous task's artifact. For example, `train` must not start unless `validate` succeeded, which is how a data problem stops a deployment. Only one branch after `decide` runs. `cleanup` depends on both branches with `none_failed_min_one_success`, because the default `all_success` rule would skip it on every run, since one branch is always skipped.

3.2 Scheduling strategy

- **Trigger types:** time-based (`@weekly`, set by the `AIRFLOW_SCHEDULE` variable), manual (after a data fix or feature change), and a planned event trigger in which Alertmanager would call the Airflow REST API after significant drift. The event trigger is not implemented in the lab.
- **Frequency:** weekly. Labels arrive with a 30–60 day delay, so daily runs would refit almost the same data. Monthly is too slow to respond to campaign-driven drift. `catchup=False` and `max_active_runs=1` prevent overlapping runs from competing for the champion alias.

3.3 Code: DAG, tasks and dependencies

```
default_args = {"owner": "mlops-team", "depends_on_past": False,
                "retries": 1, "retry_delay": timedelta(minutes=2)}

def validate(**context):
    """Quality gate on the data. Raising here stops the DAG before training."""
    ti = context["ti"]
    run_dir = Path(ti.xcom_pull(task_ids="ingest", key="run_dir"))
    report = validate_dataset(pd.read_parquet(run_dir / "raw.parquet"), raise_on_error=True)
    (run_dir / "validation_report.json").write_text(json.dumps(report, indent=2))
    ti.xcom_push(key="validation_report", value=report)

def evaluate(**context):
    ti = context["ti"]
    run_dir = Path(ti.xcom_pull(task_ids="ingest", key="run_dir"))
    split, model = joblib.load(run_dir / "split.joblib"), joblib.load(run_dir / "model.joblib")
    result = evaluate_model(model, split["X_test"], split["y_test"],
                           run_id=ti.xcom_pull(task_ids="train", key="mlflow_run_id"))
    ti.xcom_push(key="metrics", value={k: v for k, v in result.items() if isinstance(v, (int, float))})
    (run_dir / "evaluation.json").write_text(json.dumps(result, default=str)) # data -> volume

def decide(**context):
    """BranchPythonOperator: must return the task_id to run next."""
    ti = context["ti"]
    gate = passes_quality_gate(ti.xcom_pull(task_ids="evaluate", key="metrics") or {})
    ti.xcom_push(key="quality_gate", value=gate)
    return "promote_model" if gate["passed"] else "skip_promotion"

with DAG(dag_id="credit_default_training", default_args=default_args,
        schedule=os.getenv("AIRFLOW_SCHEDULE", "@weekly"), start_date=datetime(2026, 1, 1),
        catchup=False, max_active_runs=1, tags=["ml", "training", "credit-risk"]) as dag:
    t_ingest = PythonOperator(task_id="ingest", python_callable=ingest)
    t_validate = PythonOperator(task_id="validate", python_callable=validate)
    t_train = PythonOperator(task_id="train", python_callable=train)
    t_evaluate = PythonOperator(task_id="evaluate", python_callable=evaluate)
    t_decide = BranchPythonOperator(task_id="decide", python_callable=decide)
    t_promote = PythonOperator(task_id="promote_model", python_callable=promote)
    t_skip = EmptyOperator(task_id="skip_promotion")
    t_cleanup = PythonOperator(task_id="cleanup", python_callable=cleanup,
                              trigger_rule="none_failed_min_one_success")

    t_ingest >> t_validate >> t_train >> t_evaluate >> t_decide
    t_decide >> [t_promote, t_skip] >> t_cleanup
```

3.4 Operational considerations

- **Failure handling:** one retry with a 2-minute delay for transient errors, such as the MLflow server restarting. Validation errors are *not* transient, and they fail on retry too, which is intended. Tasks are idempotent because each run writes into its own `run_id` directory, and the alias change is a single atomic registry call.
- **Alerting (designed, not yet in the lab DAG):** an `on_failure_callback` should post the task, run and log URL to the team channel. The `skip_promotion` path should post the `failed_checks` list from the `quality_gate` XCom, so a rejected model is visible rather than silent. SLA misses (a run longer than 30 minutes) should also be reported.
- **Monitoring:** implemented today, the Airflow UI shows run and task duration history and MLflow compares runs over time. Planned: a StatsD exporter that sends DAG success rate and duration to Prometheus, with an alert if no successful run happens within 8 days, which would catch a paused or broken scheduler.

4. Code Quality & Documentation

4.1 Code standards

- **Implemented checks:** Lab 2 CI runs unit tests, a pipeline and registry smoke run, and an Airflow DAG parse. It does not currently enforce black, isort, flake8 or mypy.

- **Documentation:** core pipeline functions have docstrings, type hints and comments for non-obvious choices; coverage of every public function and signature has not been audited.
- **Next quality step:** add formatter, linter and type-checker checks to CI before claiming enforcement.

```
def compute_metrics(y_true: pd.Series, y_proba: np.ndarray,
                    threshold: float = REVIEW_THRESHOLD) -> Dict[str, float]:
    """Aggregate metrics at a fixed decision threshold."""
    y_pred = (y_proba >= threshold).astype(int)
    # labels=[0, 1]: a single-class slice would otherwise return a 1x1 matrix
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    return {
        "roc_auc": float(roc_auc_score(y_true, y_proba)),          # ranking metrics take the probability
        "pr_auc": float(average_precision_score(y_true, y_proba)),
        "recall": float(recall_score(y_true, y_pred, zero_division=0)), # threshold metrics take labels
        ...
    }
```

4.2 Configuration

pipeline/config.py centralises paths, feature lists, thresholds and model defaults. The tracking URI, seed and selected thresholds accept environment overrides; other values remain constants or CLI arguments. The snippet below shows settings implemented in the lab.

```
# pipeline/config.py – config loading with environment overrides
MLFLOW_TRACKING_URI = os.getenv("MLFLOW_TRACKING_URI", (BASE_DIR / "mlruns").as_uri())
MLFLOW_EXPERIMENT_NAME = os.getenv("MLFLOW_EXPERIMENT_NAME", "credit-default-risk")
RANDOM_STATE = int(os.getenv("RANDOM_STATE", 501))
MIN_ROC_AUC = float(os.getenv("MIN_ROC_AUC", 0.70))
MIN_PR_AUC = float(os.getenv("MIN_PR_AUC", 0.45))
MAX_FAIRNESS_GAP = float(os.getenv("MAX_FAIRNESS_GAP", 0.10))
MODEL_CONFIGS = {"logreg": {"C": 1.0, "max_iter": 1000},
                  "hgb": {"max_iter": 300, "learning_rate": 0.06, "max_depth": 6}}
```

5. Reproducibility & Versioning Strategy

This section is a production plan. The current labs use Git, pinned dependencies and MLflow runs; DVC data snapshots, automatic Git-SHA tagging in each run and a production deployment target have not been implemented.

Artifact	Strategy
Code	Git with GitHub Flow plus release tags : short-lived feature/* branches → pull request (CI: lint, type check, tests, pipeline smoke run) → main. Releases use semantic-version tags (v1.2.0), which trigger the CD workflow (image built and pushed to GHCR, staging, then production after approval). Every MLflow run records the git commit hash.
Data	Immutable, dated snapshots (data/snapshots/2026-09-28/credit.parquet) tracked by DVC , with the content hash stored in git and the data in remote storage. The row count, positive rate and hash are logged as run parameters, so any model can be traced back to its exact data.
Model	MLflow registry: one name per use case, automatic version numbers, aliases @champion and @challenger, and tags quality_gate, outcome and git_sha. The drift reference (reference.json) is versioned <i>with</i> the model, because it is a property of the training data.
Environment	Pinned requirements.txt. scikit-learn and MLflow versions are identical in the training image and the serving image, because a model pickled under 1.3 does not load under 1.6. Docker images are tagged with the git SHA.

```
# Code versioning – tag a release and record the commit in each run
git switch -c feature/derived-features
git commit -m "feat: add utilisation and payment ratio" # CI runs on the PR
git tag -a v1.2.0 -m "HGB champion, gap 0.028" && git push origin v1.2.0 # triggers CD
mlflow.set_tag("git_sha", subprocess.check_output(["git", "rev-parse", "HEAD"]).decode().strip())

# Random seeds – one seed from config, applied everywhere
RANDOM_STATE = int(os.getenv("RANDOM_STATE", 501))
np.random.seed(RANDOM_STATE); random.seed(RANDOM_STATE)
train_test_split(X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)
HistGradientBoostingClassifier(**params, random_state=RANDOM_STATE)
```

```
# docker/airflow.Dockerfile – training image, pinned to Airflow's constraints
FROM apache/airflow:2.8.4-python3.11
USER airflow
ARG AIRFLOW_VERSION=2.8.4
ARG PYTHON_VERSION=3.11
COPY requirements-airflow.txt /tmp/requirements-airflow.txt
RUN pip install --no-cache-dir -r /tmp/requirements-airflow.txt \
    --constraint
"https://raw.githubusercontent.com/apache/airflow/constraints-${AIRFLOW_VERSION}/constraints-${PYTHON_VERSION}.txt"
ENV PYTHONPATH=/opt/airflow/project

# Dockerfile – serving image
FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1 PYTHONPATH=/app
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt # layer cached until deps change
COPY app/ ./app/
RUN useradd --create-home --uid 1000 appuser && chown -R appuser /app
USER appuser
HEALTHCHECK CMD python -c "import json,urllib.request,sys; \
    sys.exit(0 if json.load(urllib.request.urlopen('http://localhost:8000/health'))['model_loaded'] else 1)"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Reproducing the recommended run. Check out the commit recorded in the run's `git_sha` tag, then `dvc checkout` the data snapshot whose hash is in the run parameters. Build the image from the same tag, and re-run `python -m pipeline.run_pipeline --model-type hgb --max-iter 500 --learning-rate 0.03 --max-depth 8` with `RANDOM_STATE=501`. With the same seed, data and library versions, the metrics reproduce exactly. In this work, the Lab 2 sweep and this matrix were two separate runs, and the configurations they share (for example logistic regression with $C=1.0$, and HGB with 300 iterations) gave identical metrics to four decimal places.

References: MLflow documentation, "Model Registry: aliases"; Apache Airflow documentation, "Branching" and "Trigger rules"; Huyen (2022), *Designing Machine Learning Systems*, ch. 5–6; Sculley et al. (2015).